Василенко Е.Р. 090303-ПИа-о24

Лабораторная работа №13
MongoDB и Redis: документные данные, кэширование и высокопроизводительный доступ к данным

1. Цель работы

Получить практические навыки работы с нереляционными базами данных на примере MongoDB и Redis: понять отличие документной модели данных от реляционной, освоить CRUD-операции MongoDB, вложенные объекты, массивы, фильтрацию и агрегацию, познакомиться с Redis как key-value-хранилищем, реализовать кэширование по стратегии Cache-Aside с TTL и инвалидацией, сравнить время доступа к данным через основную БД и через кэш, а также научиться выбирать подходящее хранилище в зависимости от характера данных.

2. Краткие теоретические сведения

NoSQL — общее название систем хранения данных, не использующих классическую реляционную модель. MongoDB использует документную модель: данные хранятся в коллекциях (collection) в виде документов (document) — JSON-подобных структур с полями (field). Условное соответствие SQL и MongoDB: таблица — коллекция, строка — документ, столбец — поле, первичный ключ — _id. В отличие от реляционной модели, связанные данные могут храниться непосредственно внутри документа — во вложенных объектах (embedded document) и массивах.

Основные операции MongoDB: insert_one, find_one, find, update_one ($set), delete_one. Запросы поддерживают операторы сравнения ($gte, $lt), поиск по вложенному полю (address.city) и по элементу массива (skills: "Python"). Агрегация выполняется через aggregation pipeline — последовательность стадий $match, $group, $sort, $project, $limit. Индексы создаются функцией create_index и ускоряют поиск, но увеличивают размер хранилища и стоимость записи.

Redis — высокопроизводительное хранилище типа ключ-значение, работающее в памяти. Поддерживает несколько типов данных: String, List, Set,

Hash, Sorted Set. TTL (time to live) позволяет автоматически удалять записи через заданное время — это ключевой механизм кэширования.

Cache-Aside — наиболее распространённая стратегия кэширования: при запросе сначала проверяется Redis; при попадании данные возвращаются из кэша, при промахе — читаются из основной БД и кладутся в Redis с TTL. При изменении данных выполняется инвалидация кэша (redis.delete), чтобы следующие запросы получили свежие данные. MongoDB и Redis решают разные задачи: MongoDB — основное хранение документов, Redis — быстрый доступ к часто используемым данным.

3. Индивидуальное задание

Вариант 5 — предметная область «Отель». MongoDB хранит документы room (гостиничные номера) с полями: номер, тип, этаж, характеристики (вложенный объект: площадь, вместимость, кровати), удобства (массив), цена, состояние. Redis используется для: кэша информации о номере (String), множества свободных номеров (Set), истории последних поисков (List) и статистики чтений (Hash). Требуется реализовать не менее 20 документов с вложенными объектами и массивами, CRUD, не менее 10 запросов (фильтрация, сортировка, ограничение, вложенное поле, массив, агрегация), минимум 2 индекса, минимум 3 типа данных Redis и 8 операций, кэширование с TTL и инвалидацией, benchmark (100/1000/10000 запросов) и минимум 10 автоматических тестов.

4. Проектирование MongoDB

База данных hotel_db, коллекция rooms. Документ номера имеет простые поля (number, room_type, floor, price, status), вложенный объект features (area, capacity, beds) и массив amenities. Статус number уникален. Структура документа:

```
{
  "_id": ObjectId(...),
  "number": "201",
  "room_type": "люкс",
  "floor": 2,
  "price": 6000.0,
  "features": { "area": 38, "capacity": 2, "beds": 1 },
  "amenities": ["wi-fi", "tv", "кондиционер", "мини-бар"],
  "status": "свободен"
```

```
}
```

Заполнено 20 документов: 7 стандартных, 6 люксов и 7 семейных номеров на шести этажах. Такая структура демонстрирует различия документной модели: характеристики и удобства хранятся внутри документа, и для их получения не требуется JOIN.

Поле	Тип	Описание
number	строка, unique	номер комнаты
room_type	строка	тип: стандартный / люкс / семейный
floor	число	этаж
price	число	цена за ночь, руб.
features	вложенный объект	area, capacity, beds
amenities	массив	удобства: wi-fi, tv, ...
status	строка	свободен / занят / на обслуживании

Созданы два индекса: уникальный по number — ускоряет поиск конкретного номера и гарантирует отсутствие дубликатов; составной по (room_type, price) — ускоряет фильтрацию по типу с сортировкой по цене. Индексы выбраны по реально используемым запросам.

5. Архитектура приложения

Приложение построено по схеме CLI → Service → (MongoRepository | RedisStore). Консольный интерфейс только принимает команды; бизнес-логика, cache-aside, инвалидация и benchmark находятся в HotelService; доступ к данным изолирован в MongoRepository и RedisStore. Схема приведена на рисунке 13.1.

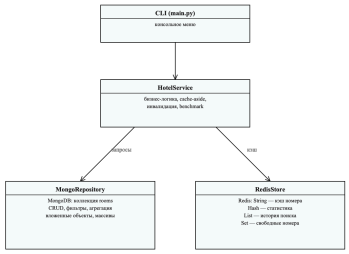


Рисунок 13.1 – Архитектура приложения

```
lab_13/
├── app/
│   ├── models.py         # Room — модель предметного объекта
│   ├── mongodb.py        # MongoRepository: CRUD, запросы, агрегация, индексы
│   ├── redis_client.py   # RedisStore: String/Set/List/Hash, TTL, кэш
│   ├── service.py        # HotelService: бизнес-логика, cache-aside, benchmark
│   └── main.py           # консольное меню из 10 пунктов
├── tests/                # conftest.py, test_app.py
├── requirements.txt
└── README.md
```

6. Реализация MongoDB

Подключение: при доступном MongoDB используется PyMongo, иначе mongomock — эмуляция MongoDB в памяти процесса (приложение работает в любом окружении). MongoRepository содержит операции CRUD, фильтрацию, сортировку с limit, запросы по вложенному полю и элементу массива, агрегацию через pipeline.

Файл: app/mongodb.py

```
from typing import Any, Dict, List, Optional

DB_NAME = "hotel_db"
COLLECTION = "rooms"


def create_mongo_client(uri: str = "mongodb://127.0.0.1:27017/", timeout: float
= 0.4):
    """Пытается подключиться к настоящему MongoDB; при недоступности
    использует mongomock — эмуляцию MongoDB в памяти процесса."""
    try:
        from pymongo import MongoClient

        client = MongoClient(uri, serverSelectionTimeoutMS=int(timeout * 1000))
        client.admin.command("ping")
        return client, "mongomock" if uri.startswith("mongomock") or "mongomock"
in uri else "pymongo"
    except Exception:
        import mongomock
```

```
        return mongomock.MongoClient(), "mongomock"


class MongoRepository:
    """Доступ к документам номеров: CRUD, сложные запросы, агрегация,
индексы."""

    def __init__(self, client, db: str = DB_NAME, collection: str = COLLECTION):
        self._collection = client[db][collection]

    def seed(self, rooms: List[Dict[str, Any]]) -> None:
        if self._collection.count_documents({}) == 0:
            self._collection.insert_many(rooms)

    def ensure_indexes(self) -> List[str]:
        return [
            self._collection.create_index("number", unique=True),
            self._collection.create_index([("room_type", 1), ("price", 1)]),
        ]

    # --- CRUD ---
    def create(self, room: Dict[str, Any]) -> str:
        result = self._collection.insert_one(room)
        return str(result.inserted_id)

    def get_by_number(self, number: str) -> Optional[dict]:
        return self._collection.find_one({"number": number})

    def list_all(self) -> List[dict]:
        return list(self._collection.find().sort("number", 1))

    def update(self, number: str, fields: Dict[str, Any]) -> Optional[dict]:
        self._collection.update_one({"number": number}, {"$set": fields})
        return self.get_by_number(number)

    def delete(self, number: str) -> bool:
        result = self._collection.delete_one({"number": number})
        return result.deleted_count > 0

    # --- фильтрация ---
    def find_by_status(self, status: str) -> List[dict]:
        return list(self._collection.find({"status": status}).sort("price", 1))
```

Сложные запросы реализованы в полном объёме: фильтрация по статусу, типу, цене ($lt), диапазону этажей ($gte/$lte), по вложенному полю features.capacity, по элементу массива amenities; сортировка по цене и номерам; ограничение количества (top_expensive); агрегация count_by_type через $group+ $sum и avg_price_by_type через $avg/$min по группам типа номера.

```
        return list(self._collection.find({"status": "свободен", "room_type":
room_type}))

    def find_cheaper_than(self, price: float) -> List[dict]:
        return list(self._collection.find({"price": {"$lt":
price}}).sort("price", 1))

    def find_by_floor_range(self, floor_min: int, floor_max: int) -> List[dict]:
        return list(self._collection.find(
            {"floor": {"$gte": floor_min, "$lte": floor_max}}
        ).sort("floor", 1))

    def find_by_feature(self, feature: str, value: Any) -> List[dict]:
        """Запрос по вложенному полю (features.*)."""
        return list(self._collection.find({f"features.{feature}": value}))
```

```
    def find_by_amenity(self, amenity: str) -> List[dict]:
        """Запрос по элементу массива amenities."""
        return list(self._collection.find({"amenities": amenity}))

    def find_capacity_at_least(self, capacity: int) -> List[dict]:
        return list(self._collection.find({"features.capacity": {"$gte":
capacity}}))

    def top_expensive(self, limit: int = 3) -> List[dict]:
        return list(self._collection.find().sort("price", -1).limit(limit))

    # --- агрегация ---
    def count_by_type(self) -> List[dict]:
        return list(self._collection.aggregate([
            {"$group": {"_id": "$room_type", "count": {"$sum": 1}}},
            {"$sort": {"count": -1}},
        ]))

    def avg_price_by_type(self) -> List[dict]:
        return list(self._collection.aggregate([
            {"$group": {"_id": "$room_type", "avg": {"$avg": "$price"}, "min":
{"$min": "$price"}}},
            {"$sort": {"_id": 1}},
        ]))
```

7. Реализация Redis

RedisStore использует минимум 3 типа данных Redis: String — кэш номера (room:101), Set — множество свободных номеров (rooms:free), List — история последних поисков (search:history), Hash — статистика чтений (stats). Реализованы операции set/get с TTL, sadd/scard/sismember, lpush/ltrim/lrange и hincrby. При недоступности Redis используется fakeredis.

Файл: app/redis_client.py

```
from typing import Any, List, Tuple


def create_redis_client(host: str = "127.0.0.1", port: int = 6379, timeout:
float = 0.4):
    """Пытается подключиться к настоящему Redis; при недоступности
    использует fakeredis — эмуляцию Redis в памяти процесса."""
    try:
        import redis

        client = redis.Redis(host=host, port=port,
socket_connect_timeout=timeout,
                             decode_responses=True)
        client.ping()
        return client, "redis"
    except Exception:
        import fakeredis

        return fakeredis.FakeRedis(decode_responses=True), "fakeredis"


class RedisStore:
    """Работа с Redis: минимум 3 типа данных (String, Hash, List, Set)."""

    CACHE_TTL = 60  # секунд; минимум по заданию — 30

    def __init__(self, client: Any):
        self._client = client
        self.hits = 0
        self.misses = 0
```

```
    # --- String: кэш номера (cache-aside) ---
    @staticmethod
    def room_key(number: str) -> str:
        return f"room:{number}"

    def get_room_cached(self, number: str):
        raw = self._client.get(self.room_key(number))
        if raw is None:
            self.misses += 1
            return None
        self.hits += 1
        return raw

    def set_room_cache(self, number: str, text: str, ttl: int = CACHE_TTL) ->
None:
        self._client.set(self.room_key(number), text, ex=ttl)

    def invalidate_room(self, number: str) -> None:
        self._client.delete(self.room_key(number))

    def cache_ttl(self, number: str) -> int:
        return int(self._client.ttl(self.room_key(number)))

    # --- List: результаты последних поисков (история) ---
    def push_search(self, query: str, limit: int = 5) -> None:
        self._client.lpush("search:history", query)
        self._client.ltrim("search:history", 0, limit - 1)

    def recent_searches(self) -> List[str]:
        return self._client.lrange("search:history", 0, -1)

    # --- Set: свободные номера ---
```

8. Кэширование (Cache-Aside) и инвалидация

Чтение номера выполняется по стратегии Cache-Aside: сначала проверяется Redis (ключ room:<number>, TTL 60 секунд); при попадании возвращается JSON из кэша, при промахе документ читается из MongoDB и помещается в кэш. Счётчики hits и misses показывают эффективность кэша. При изменении или удалении номера ключ удаляется (инвалидация), поэтому следующий запрос читает свежие данные из MongoDB.

Файл: app/service.py (фрагмент)

```
        room_type = str(data.get("room_type", "")).strip()
        if room_type not in ("стандартный", "люкс", "семейный"):
            raise ValidationError("Недопустимый тип номера")
        doc = {
            "number": number,
            "room_type": room_type,
            "floor": int(data.get("floor", 1)),
            "price": price,
            "features": data.get("features", {"area": 20, "capacity": 2, "beds":
1}),
            "amenities": list(data.get("amenities", [])),
            "status": data.get("status", "свободен"),
        }
        self._mongo.create(doc)
        self._sync_free_set()
        return doc

    # Cache-aside: сначала Redis, при промахе — MongoDB, затем запись в кэш
    def get_room(self, number: str) -> dict:
```

```
        cached = self._redis.get_room_cached(number)
        if cached is not None:
            return json.loads(cached)

        doc = self._mongo.get_by_number(number)
        if doc is None:
            raise NotFoundError(f"Номер {number} не найден")
        payload = self._clean(doc)
        self._redis.set_room_cache(number, json.dumps(payload,
ensure_ascii=False))
        self._redis.increment_reads(number)
        return payload

    @staticmethod
    def _clean(doc: dict) -> dict:
        """Возвращает документ без ObjectId — пригодный для JSON-
сериализации."""
        return {key: value for key, value in doc.items() if key != "_id"}

    def list_rooms(self) -> List[dict]:
        return self._mongo.list_all()
```

Проверка стратегии (рисунок 13.2): первый запрос — промах (misses=1, запись в кэш), второй — попадание (hits=1, TTL=60), после изменения цены кэш инвалидирован и следующий запрос снова читает MongoDB.

lab_13 — cache-aside: промах, попадание, TTL=60

```
5. Изменить номер
6. Удалить номер
7. Статистика (агрегация)
8. Состояние кэша
9. Очистить кэш
10. Тест производительности (benchmark)
0. Выход
Выберите действие: Номер комнаты: {
 "number": "101",
 "room_type": "стандартный",
 "floor": 1,
 "price": 2500.0,
 "features": {
  "area": 18,
  "capacity": 2,
  "beds": 1
 },
 "amenities": [
  "wi-fi",
  "tv"
 ],
 "status": "свободен"
}
Кэш: hits=0, misses=1, TTL=60c
========== ОТЕЛЬ (MongoDB + Redis) ==========
1. Добавить номер
2. Найти номер (кэш Redis)
3. Список номеров
4. Поиск по условиям
5. Изменить номер
6. Удалить номер
7. Статистика (агрегация)
8. Состояние кэша
9. Очистить кэш
10. Тест производительности (benchmark)
0. Выход
Выберите действие: Номер комнаты: {
 "number": "101",
 "room_type": "стандартный",
 "floor": 1,
```

Рисунок 13.2 – Cache-Aside: промах, попадание и TTL



Рисунок 13.3 – Инвалидация кэша после изменения номера

9. Эксперимент: MongoDB vs Redis

Проведено сравнение времени повторяющегося чтения одного номера через MongoDB (без кэша) и через Redis (кэш) для 100, 1 000 и 10 000 запросов. Для чистоты эксперимента кэш предварительно прогрет. Результаты приведены в таблице 13.1.

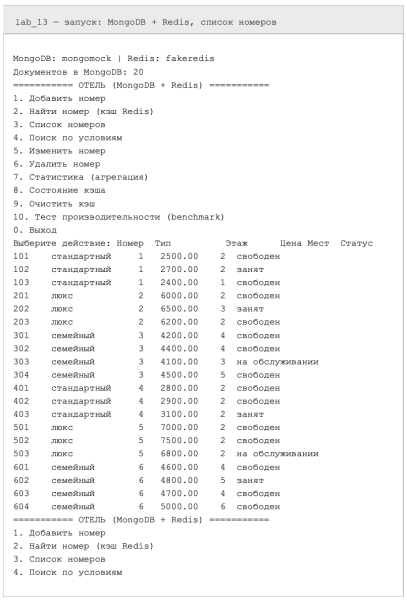
Количество запросов	**MongoDB, общее, мс**	**MongoDB, среднее, мс**	**Redis, общее, мс**	**Redis, среднее, мс**	**Ускорение**
100	5.30	0.0530	3.45	0.0345	x1.5
1 000	52.40	0.0524	31.09	0.0311	x1.7
10 000	520.72	0.0521	313.73	0.0314	x1.7

Redis быстрее MongoDB приблизительно в 1,5–1,7 раза: кэш хранит данные в оперативной памяти и отдаёт их без парсинга документов и работы индексов. Разница меньше, чем в реальной системе, потому что эмуляции (mongomock и fakeredis) работают в памяти одного процесса; при настоящих серверах разрыв между сетевым MongoDB и in-memory Redis обычно значительно больше. Важно: ускорение достигается для повторяющихся чтений

одних и тех же данных; для уникальных запросов кэш бесполезен, поэтому кэшировать следует только часто запрашиваемые данные с допустимой задержкой обновления.

10. Демонстрация работы

Тест 1. Запуск приложения: подключение к MongoDB и Redis, список номеров, поиск номера (рисунок 13.4).



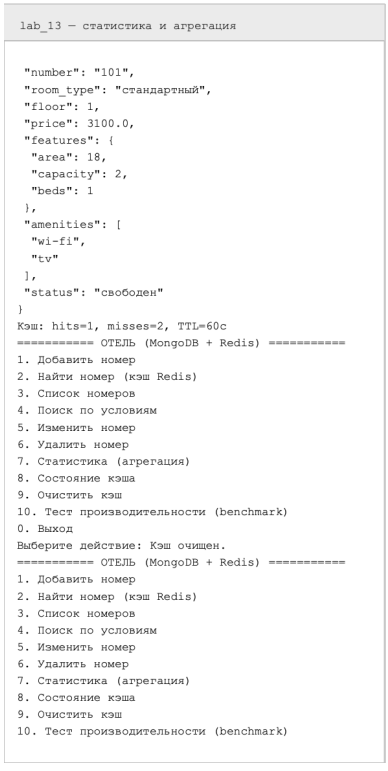
```
lab_13 — запуск: MongoDB + Redis, список номеров

MongoDB: mongomock | Redis: fakeredis
Документов в MongoDB: 20
========== ОТЕЛЬ (MongoDB + Redis) ==========
1. Добавить номер
2. Найти номер (кэш Redis)
3. Список номеров
4. Поиск по условиям
5. Изменить номер
6. Удалить номер
7. Статистика (агрегация)
8. Состояние кэша
9. Очистить кэш
10. Тест производительности (benchmark)
0. Выход
Выберите действие: Номер  Тип          Этаж      Цена Мест  Статус
101    стандартный     1   2500.00    2  свободен
102    стандартный     1   2700.00    2  занят
103    стандартный     1   2400.00    1  свободен
201    люкс            2   6000.00    2  свободен
202    люкс            2   6500.00    3  занят
203    люкс            2   6200.00    2  свободен
301    семейный        3   4200.00    4  свободен
302    семейный        3   4400.00    4  свободен
303    семейный        3   4100.00    3  на обслуживании
304    семейный        3   4500.00    5  свободен
401    стандартный     4   2800.00    2  свободен
402    стандартный     4   2900.00    2  свободен
403    стандартный     4   3100.00    2  занят
501    люкс            5   7000.00    2  свободен
502    люкс            5   7500.00    2  свободен
503    люкс            5   6800.00    2  на обслуживании
601    семейный        6   4600.00    4  свободен
602    семейный        6   4800.00    5  занят
603    семейный        6   4700.00    4  свободен
604    семейный        6   5000.00    6  свободен
========== ОТЕЛЬ (MongoDB + Redis) ==========
1. Добавить номер
2. Найти номер (кэш Redis)
3. Список номеров
4. Поиск по условиям
```

Рисунок 13.4 – Запуск и работа приложения

Тест 2. Статистика: количество номеров, агрегация по типам, средняя цена (рисунок 13.5).

lab_13 — статистика и агрегация



```
 "number": "101",
 "room_type": "стандартный",
 "floor": 1,
 "price": 3100.0,
 "features": {
  "area": 18,
  "capacity": 2,
  "beds": 1
 },
 "amenities": [
  "wi-fi",
  "tv"
 ],
 "status": "свободен"
}
Кэш: hits=1, misses=2, TTL=60c
========== ОТЕЛЬ (MongoDB + Redis) ==========
1. Добавить номер
2. Найти номер (кэш Redis)
3. Список номеров
4. Поиск по условиям
5. Изменить номер
6. Удалить номер
7. Статистика (агрегация)
8. Состояние кэша
9. Очистить кэш
10. Тест производительности (benchmark)
0. Выход
Выберите действие: Кэш очищен.
========== ОТЕЛЬ (MongoDB + Redis) ==========
1. Добавить номер
2. Найти номер (кэш Redis)
3. Список номеров
4. Поиск по условиям
5. Изменить номер
6. Удалить номер
7. Статистика (агрегация)
8. Состояние кэша
9. Очистить кэш
10. Тест производительности (benchmark)
```

Рисунок 13.5 – Статистика и агрегация MongoDB

Тест 3. Поиск по условиям, удаление номера с инвалидацией кэша и тест производительности (рисунок 13.6).

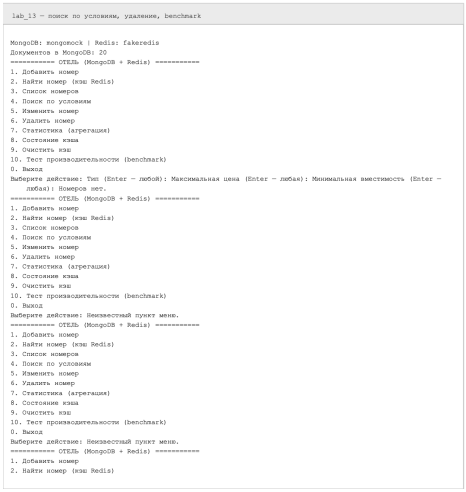


Рисунок 13.6 – Поиск, удаление и benchmark

Тест 4. Обработка ошибок: неположительная цена, дубликат номера, отсутствующий номер (рисунок 13.7).

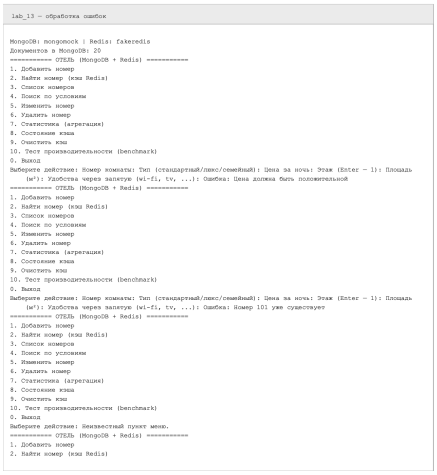


Рисунок 13.7 – Обработка ошибок приложения

11. Тестирование

Написано 21 автоматический тест pytest: CRUD MongoDB (создание, чтение, изменение, удаление, дубликат, неположительная цена), сложные запросы (фильтр по статусу, вложенному полю, массиву, limit и сортировка, агрегации), Redis (String с TTL, List, Set, Hash) и Cache-Aside (промах/попадание, инвалидация после UPDATE, benchmark). Тесты используют mongomock и fakeredis, поэтому не требуют запущенных серверов. Результат выполнения pytest приведён на рисунке 13.8.

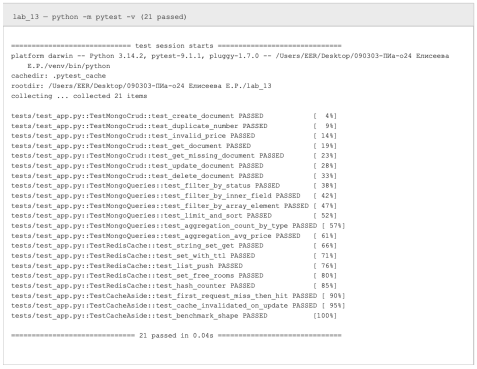


Рисунок 13.8 – Результаты выполнения pytest (21 passed)

12. Вывод

В ходе лабораторной работы разработано Python-приложение «Отель» с совместным использованием MongoDB и Redis: спроектирована документная модель номера с вложенным объектом характеристик и массивом удобств, заполнено 20 документов, реализованы CRUD, сложные запросы (фильтрация, сортировка, вложенные поля, массивы, агрегация) и индексы.

Redis использован для решения конкретной задачи — кэширования часто запрашиваемых номеров по стратегии Cache-Aside с TTL и инвалидацией; дополнительно задействованы Set для свободных номеров, List для истории поисков и Hash для статистики. Эксперимент показал ускорение повторных чтений через кэш приблизительно в 1,7 раза. Главный вывод работы: MongoDB и Redis решают разные задачи — документное хранилище и быстрый кэш, — и в реальном приложении они используются одновременно, а выбор хранилища определяется характером данных, частотой обращения и требованиями к актуальности.